
SIMULATION OF A SIMPLE COMMUNICATION SYSTEM

Communication Systems Project



Instructor : Prof. Pakravan

Moein Abdollahi Sarvi - 401100885
Winter 2025

Contents

1	Abstract	3
2	Block by Block Implementation	4
2.1	Image Pre-Processing	4
2.2	Converting to Bit Stream	5
2.3	Modulation	6
2.4	Channel	12
2.5	Noise	14
2.6	Matched Filter	15
2.7	Equalizer	17
2.7.1	Zero-Forcing Equalizer	17
2.7.2	MMSE Equalizer	19
2.8	Reconstructing Image and Post Processing	20
2.9	Pulse Shaping Effect	21
2.10	Channel Effect	22

1 Abstract

This project focuses on the simulation of a digital communication system that includes all key stages for transmitting and receiving data efficiently. The process begins with preprocessing, where input data is prepared and converted into a digital bitstream. The bitstream is then modulated, meaning the digital data is mapped into signals suitable for transmission through a communication channel. We use a polar line coding scheme, which helps improve signal representation and maintain efficient transmission.

To shape the transmitted signals, we explore two different types of pulse shaping: the half-wave sine pulse and the square root raised cosine (SRRC) pulse. These pulses play an important role in controlling bandwidth and reducing inter-symbol interference (ISI), which can otherwise cause errors in data reconstruction.

The communication channel is simulated to include real-world effects such as noise and distortions. The effect of additive white Gaussian noise (AWGN) and other channel impairments are studied to see how they impact the system's performance. At the receiver end, a matched filter is used to enhance the received signal by maximizing its signal-to-noise ratio (SNR). An equalizer is then applied to reduce further signal distortions caused by the channel.

Finally, signal reconstruction is performed to recover the original transmitted data. The simulation demonstrates how the use of pulse shaping, matched filters, and equalization improves system reliability and reduces errors. This study highlights how different channel conditions and pulse shaping methods affect the overall communication quality, providing insights into key design trade-offs in modern communication systems.

2 Block by Block Implementation

2.1 Image Pre-Processing

Goal of this part is to define a set of symbols that represent the image that is going to be communicated through channel. Image is loaded in the Python environment using `cv2.imread()`. Then the values of each pixel is converted to a double (64 bits) binary number. All values to gather form a matrix (or 3-D array if considering color image). This matrix is then divided into 8 by 8 blocks using `skimage.util.view_as_blocks()`. DCT is applied on each block separately. For each block a new DCT coefficient block is obtained. These coefficients are then scaled and quantized.

Quantization is done by 256 levels using `Numpy.round()`. As result each pixel value is represented by a 8 bit binary number. each bit is stored as a one Byte `Numpy.uint8` symbol. Finally a 3-D array is made by putting all blocks on each other.

```
image = cv2.imread('picture.jpg', cv2.IMREAD_GRAYSCALE)
def process_image(image):
    image = image.astype(np.float64)
    # Ensure the image dimensions are divisible by 8
    height, width = image.shape
    if height % 8 != 0 or width % 8 != 0:
        height = height - (height % 8)
        width = width - (width % 8)
        image = image[:height, :width]

    # Divide the image into 8x8 blocks
    blocks = view_as_blocks(image, block_shape=(8, 8))

    # Apply DCT to each block
    dct_blocks = np.zeros_like(blocks)
    for i in range(blocks.shape[0]):
        for j in range(blocks.shape[1]):
            dct_blocks[i, j] = dct(dct(blocks[i, j].T, norm='ortho').T, norm='ortho')

    # Scale DCT values to [0, 1]
    min_val = dct_blocks.min()
    max_val = dct_blocks.max()
    scaled_blocks = (dct_blocks - min_val) / (max_val - min_val)

    # Quantization
    quantization_levels = 255
    quantized_blocks = np.round(scaled_blocks * (quantization_levels))
    quantized_blocks = quantized_blocks.astype(np.uint8)

    # Reshape to 3D array (8x8xN)
    n_blocks = quantized_blocks.shape[0] * quantized_blocks.shape[1]
    reshaped_3d = quantized_blocks.reshape(n_blocks, 8, 8)
    final_3d_array = reshaped_3d.transpose(1, 2, 0)

    print(quantized_blocks.shape)
    print(n_blocks)
    return final_3d_array

final_3d_array = process_image(image)
```

Listing 1: Pre-Processing Image in Python

2.2 Converting to Bit Stream

The function `convert_to_bitstream` transforms a 3D array of quantized image blocks into a bitstream. It first reshapes each 8×8 block (64 values) into a matrix of size $64 \times \text{num_blocks}$. To efficiently store and process the data, the function groups these columns into chunks, where each chunk contains N blocks.

If the total number of blocks is not perfectly divisible by N , the function ensures that the remaining blocks are placed in a smaller chunk. Each chunk is then converted into an 8-bit binary format using `astype(np.uint8).tobytes()`, making it suitable for storage or transmission.

```
def convert_to_bitstream(final_3d_array, N):
    # Reshape the 3D array into 64 num_blocks (flattening each 8 8 block)
    num_blocks = final_3d_array.shape[2]
    flattened_blocks = final_3d_array.reshape(64, num_blocks)

    # Determine the number of chunks
    num_chunks = num_blocks // N # Number of full chunks
    remainder = num_blocks % N # Remaining blocks that don't fit in full chunks

    # Split into chunks of size 64 N
    chunks = [flattened_blocks[:, i * N: (i + 1) * N] for i in range(num_chunks)]

    # If there are leftover blocks, add them as a smaller chunk
    if remainder > 0:
        chunks.append(flattened_blocks[:, num_chunks * N:])

    # Convert each chunk to a bitstream (each value stored as 8-bit binary)
    bitstream_chunks = [chunk.astype(np.uint8).tobytes() for chunk in chunks]

    return bitstream_chunks
```

Listing 2: Converting to Bit Stream

Each chunk consists of $64 \times N$ numbers. The 64 values come from flattening each 8×8 block, and N represents the number of blocks grouped together in a chunk. These numbers are quantized values, meaning they have been compressed into a fixed set of levels. Since each quantized value is stored as 8 bits (1 Byte), the total size of a single chunk is:

$$64 \times N \text{ Bytes}$$

This ensures that the image data is structured efficiently for further processing.

2.3 Modulation

Question 2.1. Plot the time-domain function and frequency response of both pulses. Which pulse uses more bandwidth? What do these results mean? If you increase the cut-off factor (K) of the SRRC pulse, what do you expect to observe?

Two different pulse shaping is used in this simulation. Half-sine pulse and SRRC pulse. Figure 1 shows these two pulses in time domain.

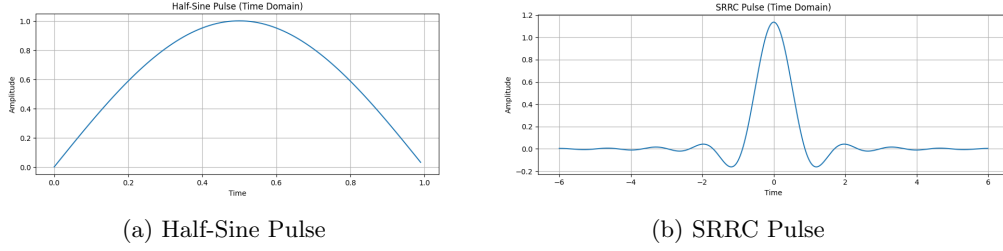


Figure 1: Half-Sine Pulse and SRRC Pulse in Time Domain

We expect that when a signal is compressed in the time domain, it will spread out more in the frequency domain. So, for smaller values of K , it uses more bandwidth. In general, comparing the two pulses, we see the usual trade-off: to reduce bandwidth, we have to allow the pulse to spread more in the time domain, which increases the chance of Inter-Symbol Interference (ISI). Figures below show the frequency domain behavior of half-sine pulse and SRRC pulse for different values of $K \in \{2, 4, 6\}$.

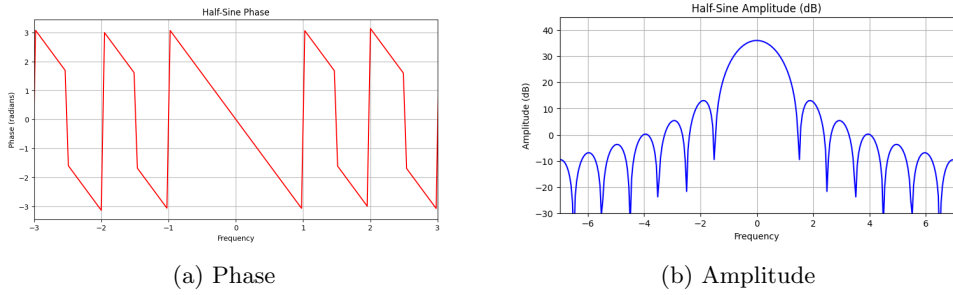


Figure 2: Frequency Response of Half-Sine Pulse

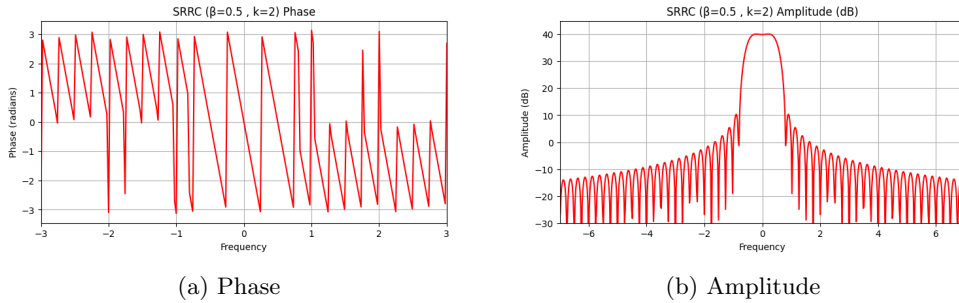


Figure 3: Frequency Response of SRRC Pulse for $K = 2$

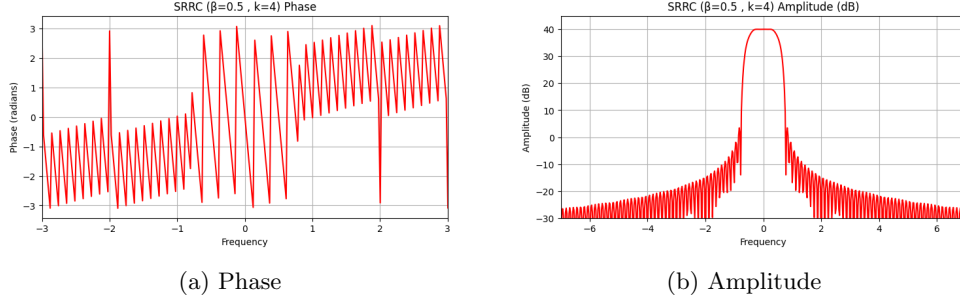


Figure 4: Frequency Response of SRRC Pulse for $K = 4$

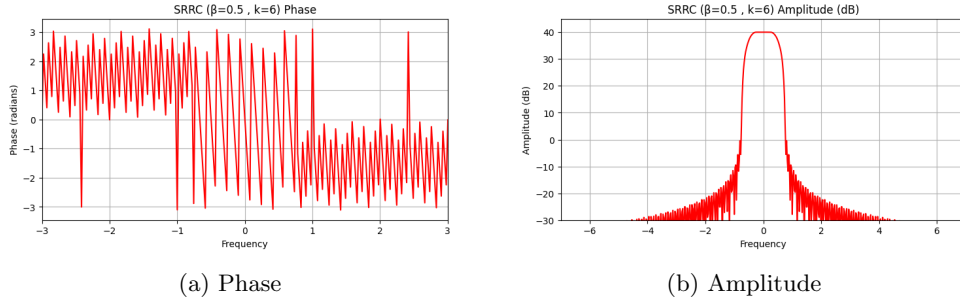


Figure 5: Frequency Response of SRRC Pulse for $K = 6$

As we expect for the least K Bandwidth is maximized. Figure 6 shows amplitude of frequency responses of these 4 pulses. As we had expect half-sine pulse (shortest in time) has more bandwidth than other pulses.

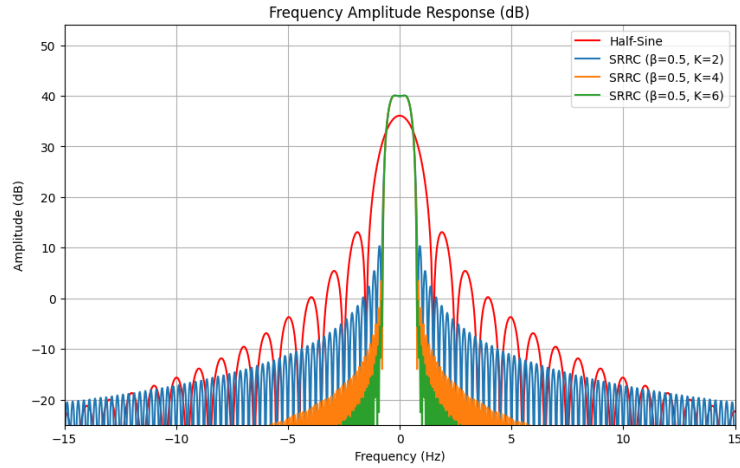


Figure 6: Comparison of Band Width of Different Pulses

As we can see SRRC pulses use less bandwidth than half-sine pulse and even as K increases, bandwidth decreases too. But the cost of this less bandwidth is that pulses are not bounded within a duration of a bit. As a result there is a chance for happening ISI.

Question 2.2. Plot the spectrum of the modulated signal (at the output of the pulse shaping filter) and compare it with the spectrum of the pulse for each pulse.

As we expect we can see in Figure 7 that the half-sine pulse uses more band width than SRRC pulse even for the least value of K which is considered as 2.

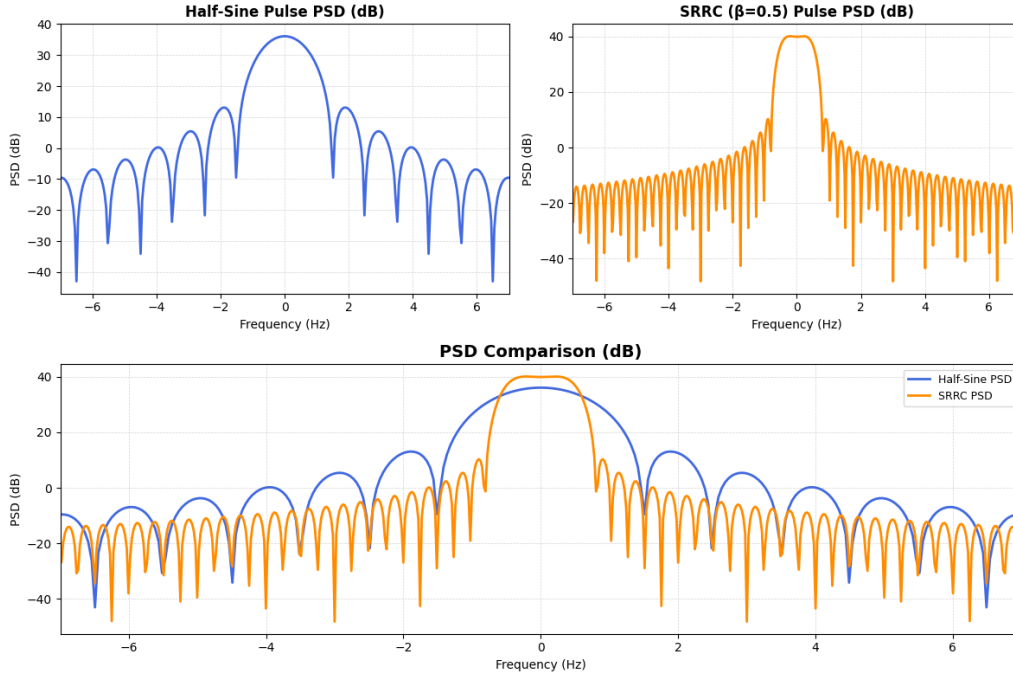


Figure 7: PSD of Polar Line Code for TWO Different Pulse Shaping

Question 2.3. For each pulse shaping function, plot the modulated signal over a time duration for 10 random bits. Explain what you observe regarding the generated signal.

Using `generate_random_bits()` 10 random bits are generated and by `polar_mappig()` those bits are converted in polar line code format.

```
def generate_random_bits(num_bits=10):
    return np.random.randint(0, 2, size=num_bits)

def polar_mapping(bits):
    return 2*bits - 1
```

Listing 3: Generate Random Bits

Polar line code numbers are amplitudes of pulses. Finally by putting these pulses beside each other a waveform is created. below time domain plot of the created wave form assigned to this set of randomly generated bits:

$$\begin{bmatrix} 0 & 1 & 1 & 0 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

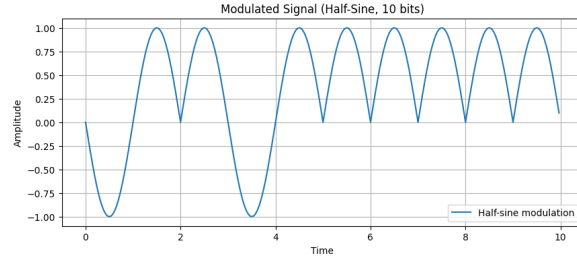


Figure 8: Generated Waveform for Half-Sine Pulse

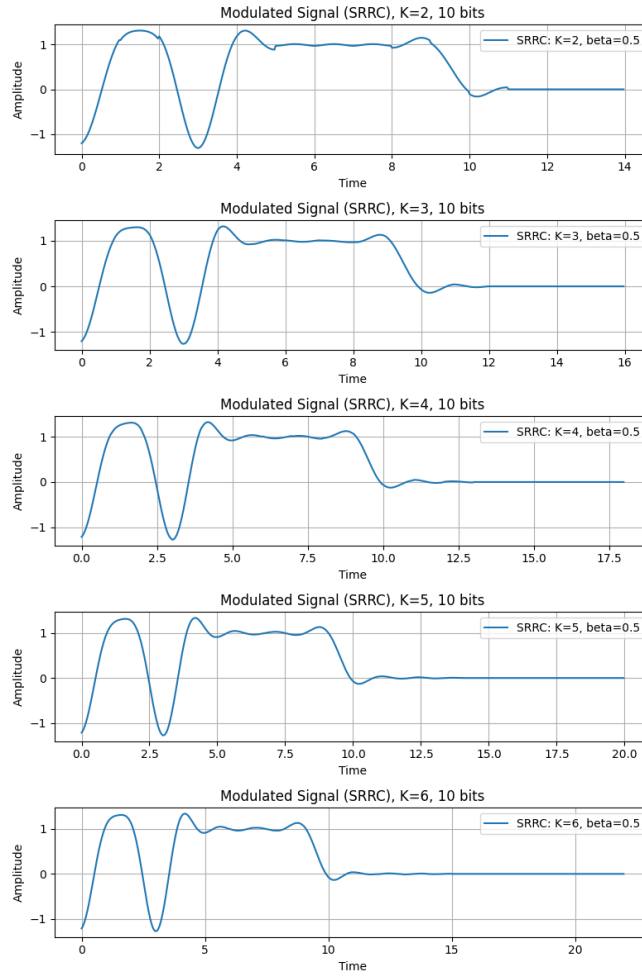


Figure 9: Generated Waveform for SRRC Pulse for Different Values of K

Question 2.4. Plot the Eye Diagram for the transmitted signal of both pulses. Do you see the eye opening? Explain what you observe.

To draw the eye diagram, I first sampled the modulated signal with high resolution (e.g., 32 samples per bit). Then, I segmented the signal into overlapping windows, each spanning one bit duration. Each window was aligned and overlaid on the same plot, creating the characteristic eye-like pattern. The x-axis represented time (within the bit duration), and the y-axis showed the amplitude of the signal. The resulting diagram revealed ISI as a result of SRRC pulse shaping.

```
def plot_eye_diagram_two_seconds(signal, sps, num_bits, title="Eye_Diagram_2-second_intervals"):
    plt.figure(figsize=(8,4))

    # We'll plot up to num_bits-1 intervals (since each 2-second slice covers 2 bits).
    for i in range(num_bits - 1):
        start = i * sps
        stop = start + 2*sps
        if stop > len(signal):
            break
        seg = signal[start:stop]
        plt.plot(np.arange(len(seg)), seg, alpha=0.8)

    plt.title(title)
    plt.xlabel("Sample_Index_within_2_bits")
    plt.ylabel("Amplitude")
    plt.grid(True)
    plt.show()
```

Listing 4: Python Code for Plotting Eye Diagram in 2 Seconds Intervals

Results are shown in Figure 10 and Figure 11. We can obviously see that ISI happened for SRRC pulse and because of that eye opening has decreased. A smaller eye opening is bad because it makes it harder to tell apart 1s and 0s in the signal. More noise and distortion can cause errors when reading the data. Timing issues (jitter) can make it difficult to sync the signal correctly. Overlapping symbols (ISI) can mix signals together and create confusion. In the end, this leads to more errors and worse communication quality.

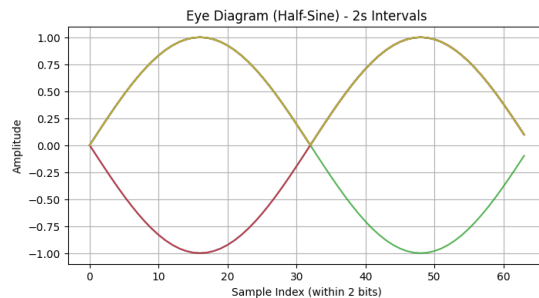


Figure 10: Eye Diagram for Half-Sine Pulse

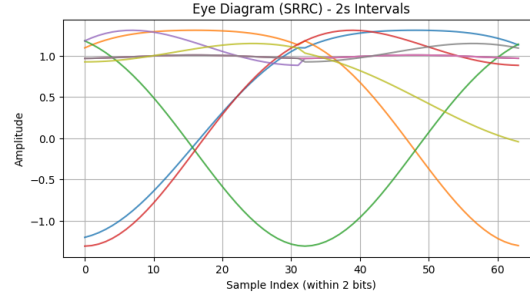


Figure 11: Eye Diagram for SRRC Pulse

Now we can see that ISI has happened in the SRRC pulse. If we want to use less bandwidth and avoid using pulses like half-sine, we must find other ways to reduce ISI. One method is equalization, where a special filter at the receiver reverses the distortion caused by ISI. The Zero-Forcing Equalizer removes ISI completely but may boost noise. The MMSE Equalizer balances ISI removal and noise reduction for better performance. Another method is Decision Feedback Equalization (DFE), which predicts and cancels ISI using past detected bits. These techniques help recover the original signal, making communication clearer and more reliable without needing wider bandwidth.

2.4 Channel

Impulse response of the channel is given as:

$$h[n] = \delta[n] + \frac{1}{2}\delta[n-1] + \frac{3}{4}\delta[n-2] - \frac{2}{7}\delta[n-3]$$

Question 2.5. Plot the frequency response (both phase and magnitude) and the impulse response of the channel.

If impulse response of channel had been obtained by sampling with the same sampling frequency of generating pulses, the impulse response and frequency response of the channel would be as Figure 12 and Figure 13.

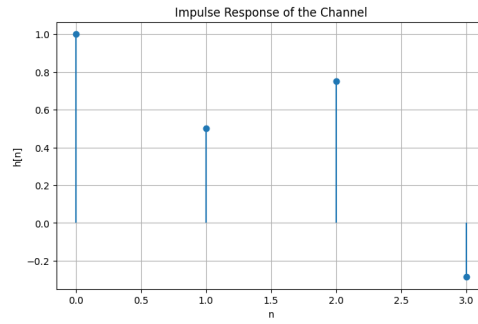


Figure 12: Impulse Response of Channel (If Sampling Frequencies Were Equal)

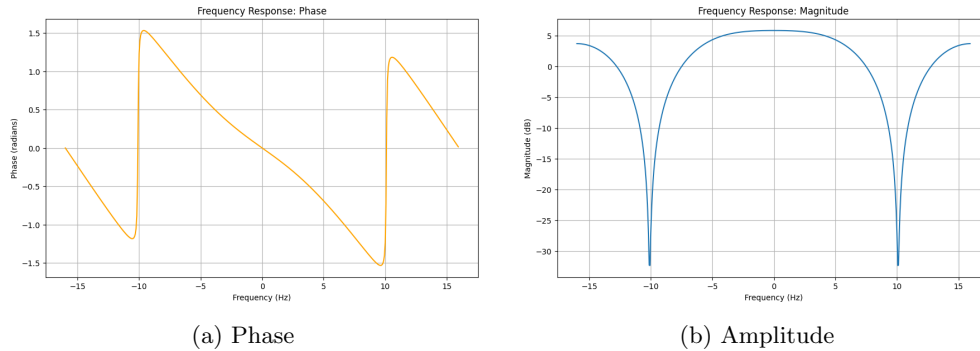


Figure 13: Frequency Response of Channel (If Sampling Frequencies Were Equal)

But the sampling frequency for sampling channel impulse response is much lower and It is in fact 1 sample per second. While sampling frequency for constructing pulses is 32 sample per second. As a result there is a need for zero-padding $h[n]$. between each two consecutive peaks 31 zeroes are added. So the real impulse response of channel and frequency response of channel is shown in Figure 14 and Figure 15.

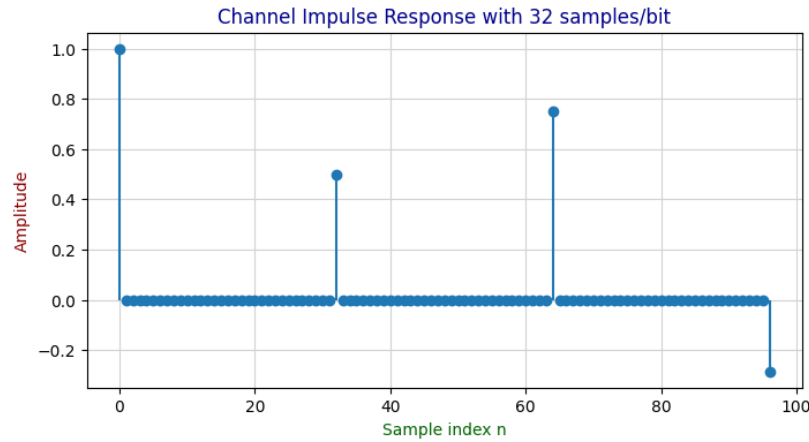


Figure 14: Impulse Response of Channel

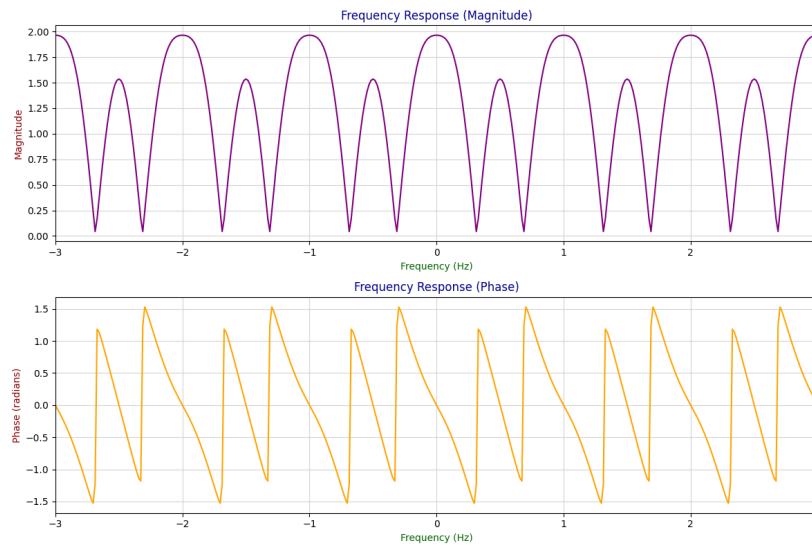


Figure 15: Frequency Response of Channel

Question 2.6. Plot the Eye Diagram of the channel output for each pulse shaping function. Explain your results and also discuss the relationship between eye opening and ISI (Inter-Symbol Interference).

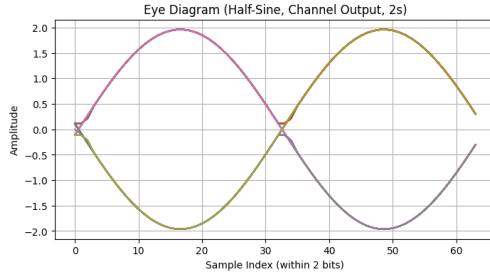
It is now explained that how a waveform is generated for a certain bit pattern and line code for each pulse. for passing that signal through channel we need to convolve the signal with channel impulse response. I used `Numpy.convolve()` for doing that. Result is then plotted in an eye diagram with the same function that was designed for the last part.

```

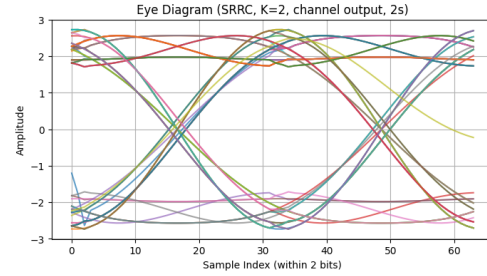
t_half, x_half = build_half_sine_transmit_signal(bits_polar, sps=sps)
# Channel output
h = get_channel_impulse_response()
y_half = np.convolve(x_half, h, mode='full')

```

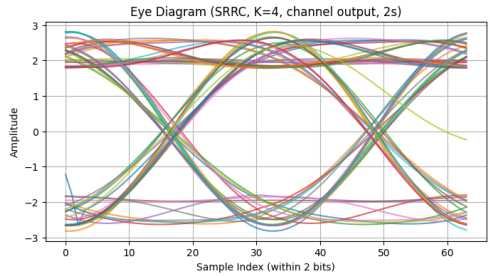
Listing 5: Passing Signal Through Channel in Python



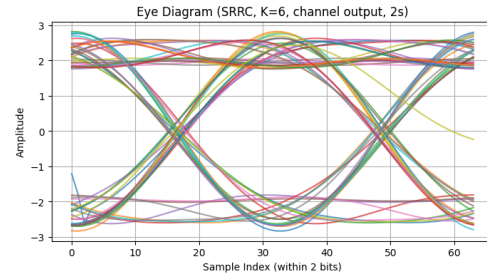
(a) Half-Sine Pulse



(b) SRRC Pulse K = 2



(c) SRRC Pulse K = 4



(d) SRRC Pulse K = 6

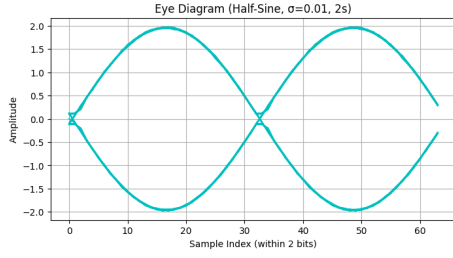
Figure 16: Eye Diagram of Channel Output for Different Pulses

In Figure 17 we can see that eye-opening decreases for SRRC pulses. Less eye opening means all patterns of 1s and 0s are not exactly like each other which one of the common reasons for that is ISI. When ISI happens pulses of one bit will affect other bits in the final waveform which causes a confusion in decision making for the receiver.

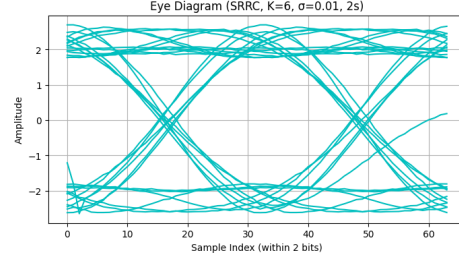
2.5 Noise

Noise is modeled using `Numpy.random.randn()` to create an array of Gaussian noise. This array is added to signal values and creates a new noisy signal.

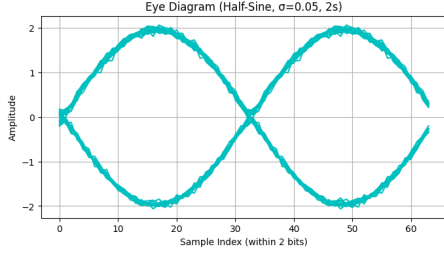
Question 2.7. Plot the eye diagrams again for the output of the channel with added noise. Explain your results. (You can experiment with different noise power values such as σ^2)



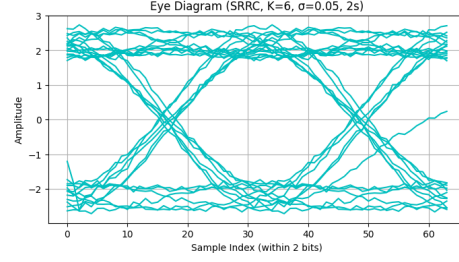
(a) Half-Sine Pulse ($\sigma = 0.01$)



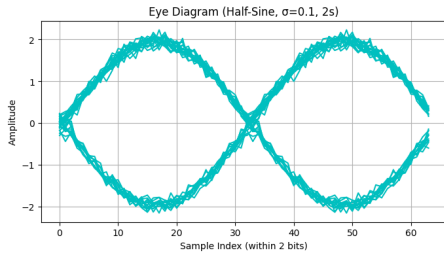
(b) SRRC Pulse ($\sigma = 0.01$)



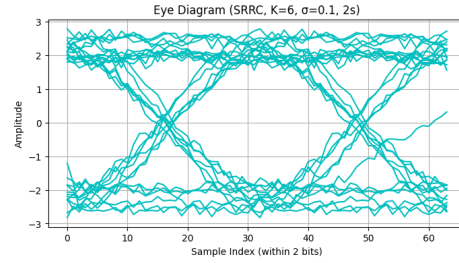
(c) Half-Sine Pulse ($\sigma = 0.05$)



(d) SRRC Pulse ($\sigma = 0.05$)



(e) Half-Sine Pulse ($\sigma = 0.1$)



(f) SRRC Pulse ($\sigma = 0.1$)

Figure 17: Eye Diagram With Presence of Noise for Different Values of σ

2.6 Matched Filter

Impulse response of matched filter is given as:

$$h_{\text{filter}}(t) = p(T - t)$$

Question 2.8. Plot the impulse response and the frequency response of the matched filter for each pulse shaping function.

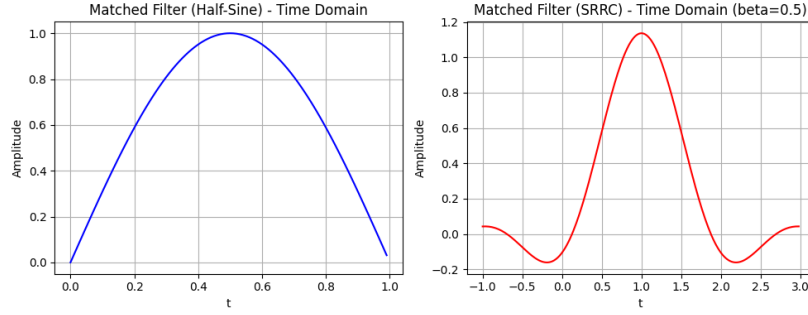


Figure 18: Matched Filter for Half-Sine Pulse and SRRC Pulse in Time Domain

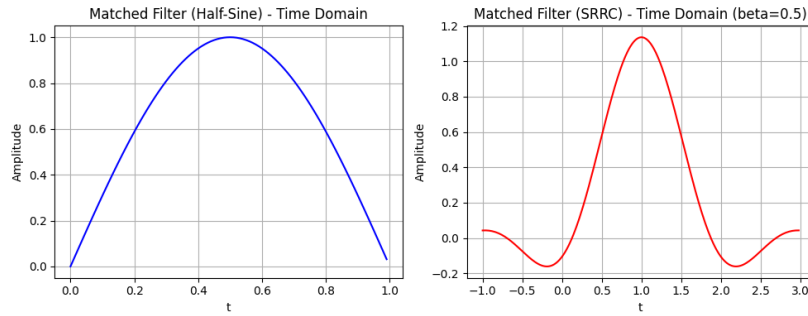


Figure 19: Matched Filter for Half-Sine Pulse and SRRC Pulse in Frequency Domain

Question 2.9. Plot the Eye Diagram for each pulse shape at the output of the matched filter. What is your best sampling point according to the Eye Diagram? Explain your results

We can see in Figure 20 and Figure 21 the eye opening has increased and comparing to Figure 17 we can observe a better eye diagram here which less effect of noise visible on it. (Although signals that were given to matched filter had noticeable value of noise $\sigma = 0.3$)

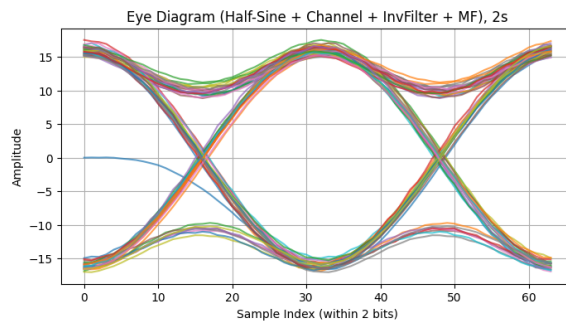


Figure 20: Eye Diagram of Half-Sine Pulse After Matched Filter

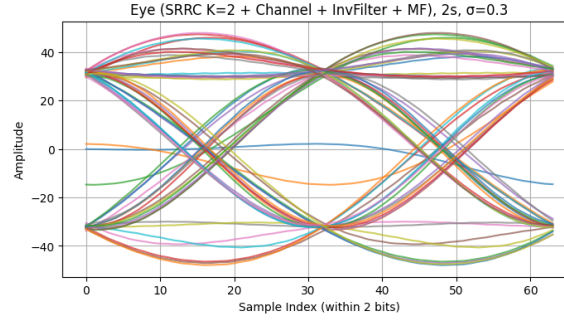


Figure 21: Eye Diagram of SRRC Pulse After Matched Filter

Best point for decision making is at maximum opening of the eye diagram. In maximum opening we have highest margin for tolerance of noise in order to make correct decision whether the incoming bit has been 1 or 0.

Also we can see that half-sine pulse has better eye diagram which is because by using half-sine pulse we used more bandwidth.

2.7 Equalizer

2.7.1 Zero-Forcing Equalizer

The ZF filter inverts the channel frequency response:

$$G_{ZF}(f) = H^{-1}(f) \quad (1)$$

Output of channel considering noise:

$$Y(f) = H(f)X(f) + N(f)$$

Applying the ZF filter:

$$\hat{X}(f) = H^{-1}(f)Y(f) = X(f) + H^{-1}(f)N(f)$$

Question 2.10. Explain how the ZF filter is implemented and draw its response. Answer and examine the frequency response and impulse response of the resulting frequency response. Is the Zero-Forcing filter stable? Can we always expect it to be the inverse of the channel?

For implementing ZF filter we can calculate Fourier transform of channel impulse response and consider inverse of that as ZF frequency response.

```
h_n = np.array([1, 1/2, 3/4, -2/7])
fs = 32

N = 512 # Number of frequency points
H_f = np.fft.fft(h_n, N) # Compute the FFT of h[n]
frequencies_Hz = np.fft.fftfreq(N, d=1/fs) # Compute frequencies in Hz

# Shift FFT output to ensure correct frequency order (negative to positive)
H_f = np.fft.fftshift(H_f)
```

```

G_f = 1 / H_f
frequencies_Hz = np.fft.fftshift(frequencies_Hz)

H_f_dB = 20 * np.log10(np.abs(H_f))
G_f_dB = 20 * np.log10(np.abs(G_f))

plt.figure(figsize=(10, 4))
plt.stem(g_n[:10])
plt.title("Impulse Response of Zero-Forcing (ZF) Filter")
plt.xlabel("n (Samples)")
plt.ylabel("g[n]")
plt.grid()
plt.show()

plt.figure(figsize=(10, 4))
plt.plot(frequencies_Hz, H_f_dB, label="|H(f)| (Original System)", color='blue',
         linewidth=2)
plt.plot(frequencies_Hz, G_f_dB, label="|G(f)| (ZF Filter)", color='red', linestyle='dashed',
         linewidth=2)
plt.title("Frequency Response: Original System vs. ZF Filter (dB Scale)")
plt.xlabel("Frequency (Hz)")
plt.ylabel("Magnitude (dB)")
plt.axhline(y=0, color='black', linestyle='--', linewidth=0.7)
plt.grid(True, which='both', linestyle='--', linewidth=0.5)
plt.legend()
plt.show()

```

Listing 6: Implementing ZF Filter in Python

As a result ZF filter is implemented.

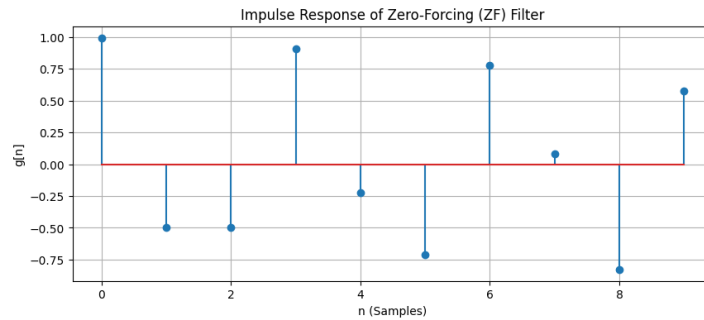


Figure 22: ZF Filter Time Domain Plot

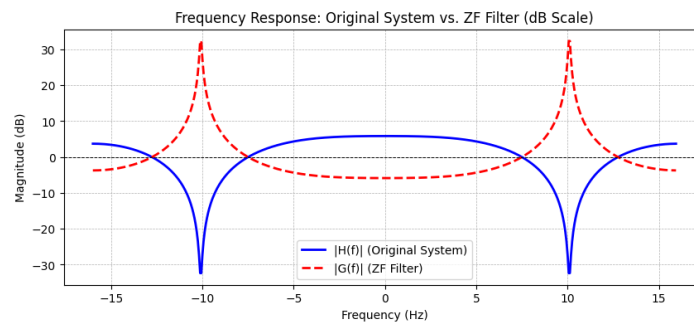


Figure 23: ZF Filter and Channel Frequency Responses

ZF filter is not necessarily stable, in frequencies which gain of the channel is 0 (completely

blocking) ZF filter limits to infinity (as we can see for this channel at $f = 10Hz$ there is a significant loss in channel so response of ZF filter is more than 30 dB there). So generally channel inverse can be unstable because gain of channel can be approximately zero at some frequencies.

Question 2.11. Draw the Eye Diagram for each pulse shape at the output of the Equalizer filter. Keep in mind that the noise level σ^2 should be indicated in your diagram. Explain your results.

As a result of using inverse of channel as a ZF filter effects of channel from eye diagram is removed. Although ZF filter can potentially even amplify noise, the resulting eye diagram of applying ZF filter is good because noise is already handled with matched filter. This makes ZF suitable in scenarios where channel conditions are known and manageable.

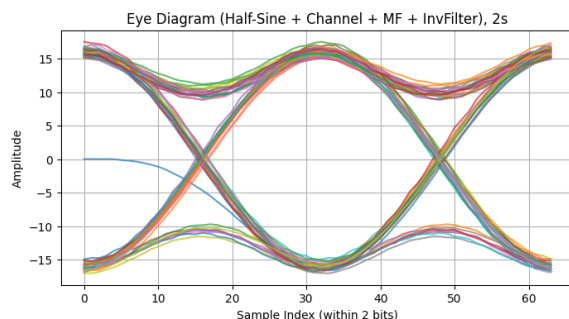


Figure 24: Output of ZF Filter for Half-Sine Pulse

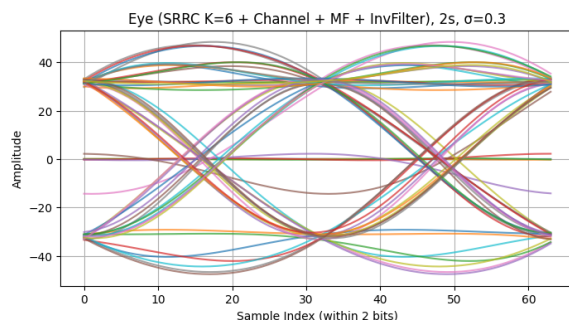


Figure 25: Output of ZF Filter for SRRC Pulse

2.7.2 MMSE Equalizer

The frequency response of the MMSE filter is given by:

$$Q_{\text{MMSE}}(e^{j\omega}) = \frac{H^*(e^{j\omega})}{|H(e^{j\omega})|^2 + \sigma^2}$$

Note that when $\sigma^2 \rightarrow 0$, the MMSE filter approaches the ZF filter, meaning that when the noise power is low, the two filters should have similar performance. However, when the noise power is high, the MMSE filter is expected to perform better than the ZF filter.

Question 2.12. Explain the method used to implement the MMSE filter. Draw the frequency response and the impulse response of this filter. Analyze the results and compare them with the results of the ZF filter.

We can implement MMSE filter using this code.

```
def get_mmse_equalizer(H, sigma=1):
    H_magnitude_squared = np.abs(H)**2
    H_conj = np.conj(H)
    Q_MMSE = H_conj / (H_magnitude_squared + sigma**2)
    return Q_MMSE
```

Listing 7: Implementing MMSE Filter in Python

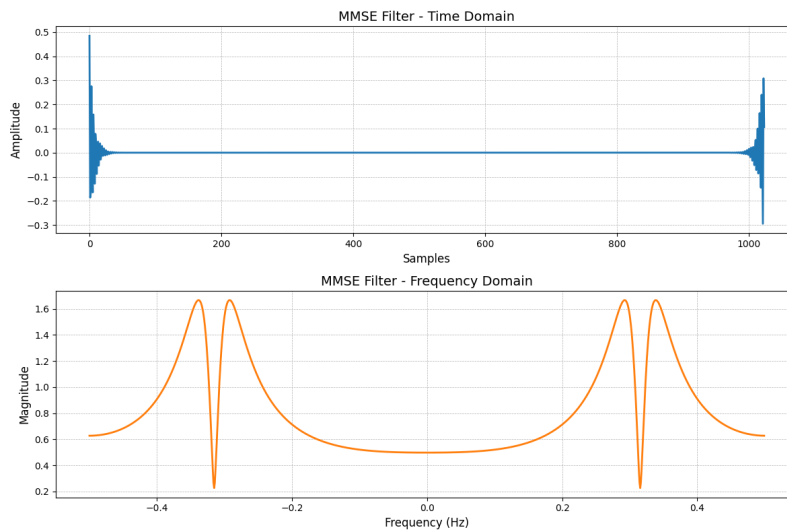


Figure 26: MMSE Filter in Time and Frequency Domain

Question 2.13. Draw the Eye Diagram for the output of the MMSE equalizer. Analyze your results.

My results are not good, but generally, we know that the MMSE filter helps in the following way. It tries to remove noise and correct the channel distortions. This makes the signal clearer and improves the eye diagram, making it more open. When the eye is more open, it's easier to detect the symbols correctly.

2.8 Reconstructing Image and Post Processing

Question 2.14. Show the result. Has your image been fully restored? Comment on what you see.

I sampled the signal at the peak points, meaning I take measurements when the signal is at its highest points. I set the decision boundary at 0 volts, which acts as a dividing line.

If the signal is above 0 volts, I considered it as one, and if it's below 0 volts, I consider it as zero. By doing this, I can determine which bit has been received, and eventually, I can reconstruct the image.

```
def sample_and_detect(received_signal, sps, num_symbols, delay):
    sampled = received_signal[delay : delay + num_symbols * sps : sps]
    detected = np.where(sampled > 0, 1, 0)
    return detected
```

Listing 8: Sampling and Decision Making

Which yields in Half-Sine BER equal to 0.619 and SRRC BER equal to 0.503.

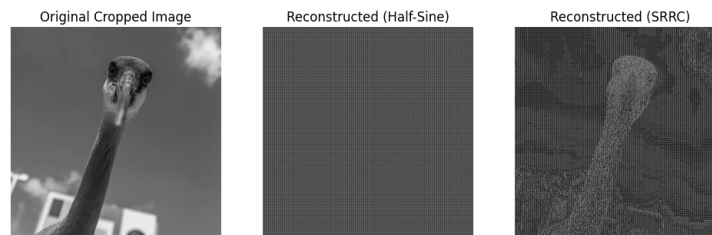


Figure 27: Comparing Transmitted Image Via Reconstructed Image

Figure 27 shows there has been a problem with implementing reconstruction of communicated image via half-sine pulse. The reason is half-sine pulse must have better accuracy in reconstruction. The designed algorithm for reconstruction is correct, since the signal can be correctly recovered when only generation of bit stream and reconstruction is done. The result is shown in Figure 28.



Figure 28: Comparing Original Image With Reconstructed Image

2.9 Pulse Shaping Effect

We implemented two different pulse shaping functions, half-sinusoidal pulses and SRRC pulses.

Question 2.15. Do these pulses satisfy the Nyquist criterion for inter-symbol interference (Zero-ISI)? At which point in the system (after the matched filter or equalizer) do you expect Zero-ISI at the sampling point? Provide your reasoning. Does your understanding align with your observations in this implementation?

The half-wave sine pulse follows the Nyquist rule, meaning it avoids inter-symbol interference (ISI) at the right sampling points. The SRRC pulse does not fully meet this rule, so it may cause some ISI. This was seen in time-domain plots, where the half-wave sine pulse had clear symbol separation, but the SRRC pulse showed some overlap.

A matched filter at the receiver boosts signal strength but does not remove ISI caused by the channel. If the transmitted pulse follows Nyquist, the filter keeps Zero-ISI at the sampling points. However, channel distortions can still cause ISI, so an equalizer is needed. Equalizers like Zero-Forcing (ZF) and MMSE help reduce ISI but must balance ISI removal with noise control.

Question 2.16. Under the same noise power and assuming that both pulse shaping functions have the same energy, do you expect any difference in error performance? Explain your reasoning. Does the simulation match your expectations? In your report, include an image of the output for each pulse shaping function under the same noise level.

Yes, the half-wave sine pulse is better because it does not cause ISI (Inter-Symbol Interference). Based on the eye diagrams, it has a wider eye opening, which means the signal is clearer and easier to read. In a real system, this gives more margin for error, making it more reliable. As a result, the system can make more accurate decisions with a higher probability. However, in practice, the main drawback is that it requires a lot of bandwidth, which makes it less efficient for real-world applications.

Question 2.17. Compare and analyze the effects of these pulse shaping functions on transmission bandwidth. Discuss the impact of pulse duration on the number of time periods and how it affects system performance.

As mentioned before, the half-wave sine pulse is more compact in the time domain, which makes it wider in the frequency domain and results in higher bandwidth consumption, which is not ideal. The more we can use time-domain extended pulses, even those that exceed the duration of a single bit, the better—as long as we can effectively control ISI for them. Overall, we want to avoid unnecessary bandwidth consumption because the more bandwidth we can use efficiently, the shorter bit durations we can transmit. As a result, this leads to an increased bitrate, which improves data transmission speed.

Question 2.18. Based on all the above points, which pulse shape is better? Also, analyze any disadvantages it may have.

In practice, pulses like SRRC or Sinc are used more often because they have a narrower bandwidth in the frequency domain. The half-sine pulse, while having some advantages like no ISI and better noise resistance, is not commonly used in real-world systems.

2.10 Channel Effect

After completing the full simulation system, test two other channels with the given impulse responses below. These are real models of wireless channels used in the design of cellular

Feature	Half-Sine Pulse	SRRC Pulse
Bandwidth Usage	High	Low
Noise Resistance	High	Low
ISI	No ISI	Has ISI
Eye Diagram Opening	Wider	Narrower

Table 1: Comparison of Half-Sine Pulse and SRRC Pulse

systems. An open-space channel is given as follows:

$$h_1[n] = 0.5\delta[n] + \delta[n-1] + 0.63\delta[n-3] + 0.25\delta[n-8] + 0.16\delta[n-12] + 0.1\delta[n-25].$$

And a closed-space channel is given as follows:

$$h_2[n] = \delta[n] + 0.4365\delta[n-1] + 0.1905\delta[n-2] + 0.0832\delta[n-3] + 0.01585\delta[n-5] + 0.003\delta[n-7].$$

There are long delays in these channels, especially in the open-space channel. After plotting the frequency response of these channels, we are able to identify where the channel experiences significant attenuation (which is referred to as "deep fading") and observe the difference between the response of the open-space and closed-space channels.

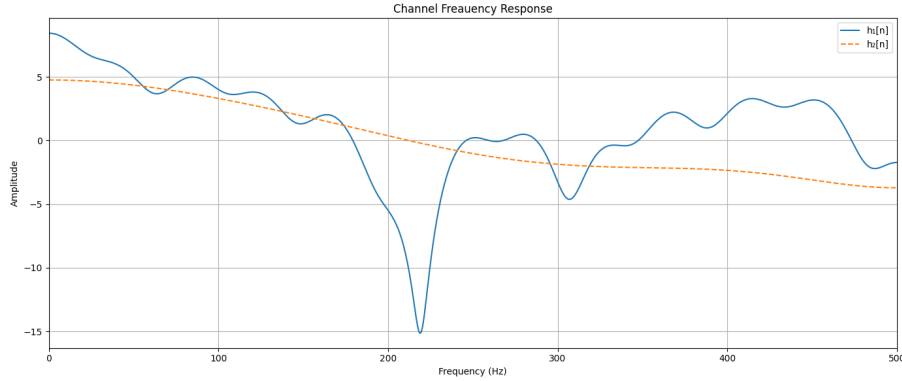


Figure 29: Frequency Response of Two Real Channels

As we can see in Figure 29 around $f = 220$ Hz we can see deep fading that is for open-space channel.